Understanding Query Likelihood

A beginner's guide to our language models experiment

Prepared for Duk Pagnarith and Team 4

2026-11-03

What this guide will help you understand

Our presentation contains several new terms at once: preprocessing, TF-IDF, MLE, Jelinek-Mercer, Dirichlet, P@3, Recall@3, MAP, and MRR. It is reasonable to need the basic ideas before discussing the experiment.

We will build those ideas step by step. Each explanation starts with an ordinary question, then uses a small example, and finally connects to our code.

Our main question: How can a computer decide which document to show first when someone types a search query?

How to study

1. Read pages 2-6 to understand the problem and query likelihood.
2. Read pages 7-9 to understand smoothing and the TF-IDF comparison.
3. Read pages 10-12 to understand evaluation and our actual results.
4. Use page 13 to connect the ideas to Python.
5. Try the questions on page 14 before reading the answers on page 15.
6. Keep page 16 nearby as a glossary and reference list.

Please pause at each **Check your understanding** question. Explain the answer in your own words before looking at the next page. A calculator is enough for the worked examples.

One sentence to keep in mind: We estimate word probabilities for each document, then rank documents by how likely their models are to generate the query.

This is a personal study document. The three-document example below is separate from the 15-document experiment in our presentation.

1. What the experiment table means

A search query is the text a user types, such as `search data`. A document is one item we might retrieve: a web page, an article, or a short text. A collection is the set of all searchable documents. A ranking is an ordered list of results.

Our experiment table describes four different decisions:

Row in the slide	Meaning in ordinary language
Preprocessing	How we prepare text before counting words
Baseline	A comparison method: TF-IDF cosine similarity
Language models	Three ways to estimate probabilities for query likelihood
Evaluation	How we check whether relevant documents appear near the top

Did we use the query likelihood model?

Yes. MLE, JM (0.2), and Dirichlet (20) all feed word probabilities into the same query-likelihood ranking idea. They differ in how they estimate those probabilities. TF-IDF is the separate comparison method.

Method label	What it means in our experiment
MLE	Query likelihood using observed document word frequencies
JM (0.2)	Query likelihood with Jelinek-Mercer smoothing; background weight 0.2
Dirichlet (20)	Query likelihood with Dirichlet smoothing; strength parameter 20
TF-IDF	Weighted word vectors compared using cosine similarity

A useful distinction

A **retrieval method** assigns scores and orders documents. An **evaluation measure** judges that ordering against relevance labels. MAP and MRR do not retrieve documents. They help us assess the results.

Check your understanding: Which is the ranking method: Dirichlet query likelihood or Recall@3? Which is the evaluation measure?

2. Preprocessing and bag-of-words

A computer needs a consistent way to turn text into countable pieces. A **token** is one piece of text produced by tokenization. In our experiment, tokens are lowercase sequences of English letters.

The exact preprocessing we use

Input: `Search, SEARCH data!`

Step	Result
Convert to lowercase	`search, search data!`
Extract alphabetic words	`search`, `search`, `data`
Count the words	search: 2; data: 1

Our tokenizer uses `re.findall(r"[a-z]+", text.lower())`. It removes punctuation from these tokens and does not keep numbers. It also splits an apostrophe or hyphen into separate alphabetic runs.

No stemming means we do not reduce word forms to a shared root. For example, `model` and `models` remain different tokens. We also keep stop words such as `the` and `of`. This tokenizer is an English teaching choice. Khmer text needs an appropriate word-segmentation strategy.

Our running collection

We will use these exact three documents on pages 3-9 and 13:

Document	Text	Token count
A	`search search data`	3
B	`search data data data`	4
C	`data science`	2

The **vocabulary** contains the distinct words: `search`, `data`, and `science`. A bag-of-words representation records how often each word appears:

Document	search	data	science
A	2	1	0
B	1	3	0
C	0	1	1

The order disappears. `search data search` has the same counts as A.

Check your understanding: What is the length of B? Does it have four vocabulary words, or four tokens?

3. Probability and a language model

A probability describes how likely an event is under a model. It ranges from 0 to 1. A value of 0.5 means one half. It does not mean 50 occurrences.

Imagine drawing a word from a bag

Document A contains three word slips: `search`, `search`, and `data`. If we randomly select one slip, two of the three slips say `search`.

$$P(\text{search} \mid A) = \frac{2}{3}, \qquad P(\text{data} \mid A) = \frac{1}{3}.$$

Read the vertical bar as **given**: “probability of search, given document A.” We have fixed A as the source from which the word is drawn.

These probabilities add to 1. No slip says `science`, so its probability under this simple model is 0.

What makes this a language model?

A language model assigns probabilities to words or sequences of words. Our small document model describes a probability distribution over the vocabulary. We build a separate distribution for each document.

Model	P(search)	P(data)	P(science)
A	2/3	1/3	0
B	1/4	3/4	0
C	0	1/2	1/2

This is a **unigram model**: each word probability is independent of previous words. A bigram model instead uses one previous word, such as $P(\text{data} \mid \text{search})$.

In Lecture 2, bigrams retain local word order. Here we choose unigrams to keep probability estimation simple. A chat model also models language, but our count-based model only needs word frequencies to score a query.

Check your understanding: Why is P(data given B) 3/4 rather than 3/9? Which source are we drawing from?

4. Query likelihood and MLE

Our query is `search data`. Query likelihood asks:

> If this document's model randomly produced words, how likely would it be to produce the words in our query?

The document does not literally write a query. This is the probability model we choose to calculate a ranking score.

Step 1: estimate word probabilities

MLE means *maximum likelihood estimation*. In this unigram count model, it gives the observed proportion of a word inside a document:

$$P_{\text{MLE}}(w \mid d) = \frac{c(w, d)}{|d|}.$$

Here w means a word, d a document, $c(w, d)$ its word count, and $|d|$ the document's number of tokens. The word "maximum" does not mean choosing only the document's most frequent word. We estimate a probability for every word.

Step 2: combine the query words

Under our independence assumption, multiply the probabilities:

$$P(\text{search data} \mid d) = P(\text{search} \mid d)\, P(\text{data} \mid d).$$

Document	Calculation	Query likelihood
A	(2/3) × (1/3)	0.2222
B	(1/4) × (3/4)	0.1875
C	0 × (1/2)	0

Step 3: rank from largest to smallest

The ranking is **A, B, C**. A gives the full query a higher probability than B, even though B contains more instances of `data`.

A high query likelihood is a scoring signal. It is not a guarantee that the document answers the user's need. Also, 0.2222 is **not** a 22.22% probability that A is relevant. It is the modeled probability of these query words given A.

Check your understanding: For query `data`, would A still rank above B?

5. Why we use logs and smoothing

Two separate problems appear when we multiply word probabilities.

Problem 1: very small products

For a long query, multiplying many probabilities can produce a number too small for a computer to represent accurately. We therefore add **natural logarithms**:

$$s(d, q) = \sum_{i=1}^{m} \log P(w_i \mid d).$$

Here the query has m tokens. If a word appears twice, it contributes twice. The logarithm preserves the ranking because it increases with its input.

Document	Query likelihood from page 5	Natural log score
A	0.222222	−1.5041
B	0.187500	−1.6740
C	0	negative infinity

Higher is better. −1.5041 is higher than −1.6740. The ranking is still A, B, C. A negative log score is normal. It does not mean a negative probability.

Problem 2: one missing query word

C contains `data`, but it has no `search`. The zero for `search` makes the entire product zero. A logarithm does not fix this. The log of zero has no finite value, and the code represents it as negative infinity.

Smoothing changes the word probabilities before we calculate the score. It gives some probability to words missing from a document.

For collection-based smoothing, we first count all three documents together:

Word	Collection count	Collection probability
search	3	3/9 = 1/3
data	5	5/9
science	1	1/9

The collection has **9 tokens**, not 3. Its vocabulary has **3 distinct words**. We write this background probability as $P(w \mid C_{all})$.

Check your understanding: Does taking logs make C's missing-word probability positive? What step do we need first?

6. Jelinek-Mercer smoothing

Jelinek-Mercer, abbreviated **JM**, mixes two distributions: the document's observed word probabilities and the collection's background probabilities.

$$P_{\text{JM}}(w \mid d) = (1 - \lambda)P_{\text{MLE}}(w \mid d) + \lambda P(w \mid C_{all}).$$

In our code, λ is the **background weight**. With $\lambda = 0.2$, we use 80% document probability and 20% background probability. Some sources define the symbol the other way around, so always check the formula.

A missing word receives background support

For document C and word `search`:

$$P_{\text{JM}}(\text{search} \mid C) = 0.8(0) + 0.2(1/3) = 0.0667.$$

For word `data`:

$$P_{\text{JM}}(\text{data} \mid C) = 0.8(1/2) + 0.2(5/9) = 0.5111.$$

The query likelihood is now approximately $0.0667 \times 0.5111 = 0.0341$. C can receive a usable score even though it misses one query word.

Results for all three documents

Document	Smoothed P(search)	Smoothed P(data)	Query likelihood
A	0.6000	0.3778	0.2267
B	0.2667	0.7111	0.1896
C	0.0667	0.5111	0.0341

The ranking remains A, B, C. **Smoothing can improve probability estimates without changing the ranking of a particular query.**

If λ moves toward 1, the models rely more on the same collection background and become less different. JM applies the same mixture weight to every document, regardless of length. The notebook's label **JM (0.2)** refers to this background weight, not to a measured accuracy of 0.2.

Check your understanding: What does $\lambda = 0.2$ mean in ordinary words?

7. Dirichlet smoothing

Dirichlet smoothing also combines document evidence with the background. It adds imaginary supporting counts, called **pseudo-counts**.

$$P_\mu(w \mid d) = \frac{c(w,d) + \mu P(w \mid C_{all})}{|d| + \mu}.$$

Read it as: **observed count plus background support, divided by observed length plus total added support**. The parameter μ is positive. Fractional pseudo-counts are allowed.

A small worked example

Use $\mu = 3$ **for this teaching example**. The query is still `search data`. C has length 2. The background probabilities are 1/3 for `search` and 5/9 for `data`.

$$P_3(\text{search} \mid C) = \frac{0 + 3(1/3)}{2+3} = \frac{1}{5} = 0.2.$$

$$P_3(\text{data} \mid C) = \frac{1 + 3(5/9)}{2+3} = \frac{8}{15} = 0.5333.$$

So $P(\text{search data} \mid C) = 0.2 \times 0.5333 \approx 0.1067$.

Document	Smoothed P(search)	Smoothed P(data)	Query likelihood
A	0.5000	0.4444	0.2222
B	0.2857	0.6667	0.1905
C	0.2000	0.5333	0.1067

What does Dirichlet (20) mean in our slide?

Our **actual notebook uses $\mu = 20$**, not 3. It is a strength parameter, not a number of documents or a probability. The background's effective weight is $\mu/(|d| + \mu)$. With $\mu = 20$ and document length 10, it is $20/30 = 0.667$. With length 100, it is $20/120 = 0.167$. Short documents receive more smoothing.

JM fixes the mixture weight directly. Dirichlet lets the weight depend on length. Neither learns synonyms. Also, the lecture's **add-one** smoothing uses one pseudo-count per vocabulary word, while Dirichlet uses collection-based support.

Check your understanding: At a fixed length, does a larger μ give more or less weight to the background?

8. Our baseline: TF-IDF cosine similarity

A **baseline** is a comparison method. We compare query likelihood with TF-IDF to see whether their ranked lists differ on the same queries and documents.

TF means term frequency: how often a word appears inside a document. **IDF** means inverse document frequency: how widely that word occurs across documents. A word found almost everywhere usually helps distinguish documents less than a word found in only a few documents.

The particular version used in our code

Scikit-learn uses smoothed IDF here:

$$\mathrm{idf}(w) = \log \frac{1 + N}{1 + \mathrm{df}(w)} + 1.$$

N is the number of documents. $\mathrm{df}(w)$ counts **documents that contain the word**, not all occurrences of the word. We multiply raw TF by IDF, then normalize each vector to length 1.

Word	Documents containing it	IDF in our three-document example
search	2	1.2877
data	3	1.0000
science	1	1.6931

A's weighted vector, ordered as search/data/science, is approximately $(2.5754, 1, 0)$. The query `search data` has vector $(1.2877, 1, 0)$ before normalization.

Cosine similarity measures how closely two vectors point in the same direction. Its formula divides their dot product by their lengths. We already normalize the vectors, so our code can simply use their dot product.

Method	What produces the score?	Better score
TF-IDF cosine	Similarity between weighted vectors	Larger cosine
Query likelihood	Probability of query words under a document model	Larger likelihood or log likelihood

Both methods use word counts. Their numerical scores have different meanings. Compare their rankings and relevance metrics, rather than comparing a cosine of 0.7 directly with a log score of −5.

Check your understanding: `search` occurs twice in A. Does that count twice toward its document frequency?

9. Relevance, P@3, Recall@3, and RR

Before evaluating, we need a judgment about which documents answer the query. These are **relevance labels**. A document containing a query word can still be irrelevant to the user's intended need.

For this separate evaluation example, suppose **four documents are relevant** in an eight-document collection. The ranking is:

Rank	Document	Relevant?
1	D1	Yes
2	D2	No
3	D3	Yes
4	D4	No
5	D5	No
6	D6	Yes
7	D7	No
8	D8	Yes

Precision at 3: how useful are the first three?

$$P@3 = \frac{\text{relevant results in the first } 3}{3} = \frac{2}{3} \approx 0.667.$$

Two of the three results we showed are relevant.

Recall at 3: how much of the relevant material did we find?

$$\text{Recall } @3 = \frac{\text{relevant results in the first } 3}{\text{all relevant documents}} = \frac{2}{4} = 0.5.$$

We found half of the relevant documents within the first three results.

Reciprocal rank: how soon is the first useful result?

The first relevant document is at rank 1, so **RR = 1/1 = 1**. If the first relevant result appeared at rank 4, RR would be 1/4 = 0.25. If none appears, RR is 0. Rank 1 is the top position in the list.

Check your understanding: Could P@3 be 1/3 while Recall@3 is 1? Under what number of relevant documents would that happen?

10. Average precision, MAP, and MRR

P@3 looks only at the first three positions. **Average precision (AP)** rewards putting all the relevant documents early in the ranked list.

Calculate AP for the previous page

Only stop at ranks where a relevant document appears. At each such rank, calculate precision using all results up to that position.

Relevant rank	Relevant results found so far	Precision at that rank
1	1	1/1 = 1.0000
3	2	2/3 = 0.6667
6	3	3/6 = 0.5000
8	4	4/8 = 0.5000

Average these contributions over the **four relevant documents**:

$$\mathrm{AP} = \frac{1 + 2/3 + 3/6 + 4/8}{4} \approx 0.6667.$$

If a relevant document is never retrieved, it contributes zero but still counts in the denominator. Our notebook calculates AP over the full 15-document ranking, including its tied tail. AP here is not AP@3.

MAP averages AP across queries

Suppose Query 1 has AP = 0.6667, and Query 2 has AP = 0.5.

$$\mathrm{MAP} = \frac{0.6667 + 0.5}{2} \approx 0.5833.$$

MAP means *mean average precision*. First calculate AP for each query, then average those AP values. It is not the same as averaging P@3.

MRR averages reciprocal ranks across queries

Suppose Query 1′s first relevant result is at rank 1, and Query 2′s is at rank 2.

$$\mathrm{MRR} = \frac{1/1 + 1/2}{2} = 0.75.$$

MRR means *mean reciprocal rank*. It considers the first relevant result for each query. MAP also considers later relevant results. A method can have MRR = 1 and still miss relevant documents from its top three.

Check your understanding: Can two rankings have the same RR but different AP? Think about what happens to relevant documents after the first one.

11. Reading our actual notebook results

We now return to the project experiment: **15 handwritten documents and 5 queries**. This is a different collection from our three-document teaching example.

Method	Mean P@3	Mean Recall@3	MAP	MRR
TF-IDF	0.600	0.950	0.900	0.900
MLE	0.333	0.500	0.619	0.815
JM (0.2)	0.600	0.950	1.000	1.000
Dirichlet (20)	0.600	0.950	1.000	1.000

Why does Recall@3 equal 0.950 rather than 1?

Q1 has **four relevant documents**, but only three fit into the first three positions. TF-IDF and both smoothed models find three relevant documents there, so Recall@3 = 3/4 = 0.75. Each has Recall@3 = 1 on the other four queries.

The mean is $(0.75 + 1 + 1 + 1 + 1)/5 = 0.95$.

Why do MAP and MRR improve?

For Q5, `car coverage`, the intended relevant document D13 discusses automobile insurance. D14 discusses a car park and repeats `car`.

- TF-IDF puts D14 first and D13 second. With one relevant document, AP = RR = 1/2 = 0.5.
- Both smoothed query-likelihood methods put D13 first, giving AP = RR = 1.
- TF-IDF has AP = RR = 1 on the other four queries. Its mean is $(1 + 1 + 1 + 1 + 0.5)/5 = 0.9$.

Same top-three relevance, different order: that is why P@3 and Recall@3 stay the same while MAP and MRR improve.

What these results do not establish

The collection is small and its relevance labels are our own judgments. MLE has many zero-probability ties, which the code orders by document ID. Its scores therefore partly reflect that arbitrary tie order. These results help us explain behavior. They do not establish that query likelihood is generally better.

Check your understanding: Does MAP = 1 mean all 15 documents are relevant?

12. Connecting the ideas to Python

This short example uses our **three-document learning collection**, with Dirichlet $\mu = 3$. It follows the same scoring steps as the notebook. The query words both occur in the collection.

```
import math
from collections import Counter

documents = {
    "A": "search search data",
    "B": "search data data data",
    "C": "data science",
}
counts = {}
background = Counter()
for name, text in documents.items():
    counts[name] = Counter(text.split())
    background.update(counts[name])

total_tokens = sum(background.values())  # 9
mu = 3
query = ["search", "data"]
scores = []
for name, word_counts in counts.items():
    length = sum(word_counts.values())
    score = 0.0
    for word in query:
        p_background = background[word] / total_tokens
        p_word = (word_counts[word] + mu * p_background)
        p_word = p_word / (length + mu)
        score = score + math.log(p_word)
    scores.append((name, score))

scores.sort(key=lambda item: item[1], reverse=True)
print(scores)  # A first, then B, then C
```

`Counter` counts words. The background combines all document counts. The inner loop calculates one smoothed probability per query token and adds its log. The last line orders documents from the largest score to the smallest.

In the actual notebook, `experiment.rank(query, method="dirichlet", mu=20)` uses the same idea with 15 documents. `method="mle"` removes smoothing; `method="jm"` uses the fixed background mixture. `method="tfidf"` uses cosine.

Try it: Change the query to `["data"]`. Which document do you expect first?

13. Practice before the presentation

Please work through these questions using a calculator or paper. The goal is to explain each step clearly, rather than memorize the formula.

A. Words and probabilities

1. Tokenize `DATA, data science!` using our preprocessing. What are the counts?
2. Document B is `search data data data`. What are its length, P(search given B), and P(data given B) under MLE?
3. For query `data data`, what is B's MLE query likelihood? Why does `data` appear twice in the calculation?
4. For query `search data`, why does document C receive MLE likelihood zero? Does taking logs solve the problem?

B. Smoothing

5. The collection has 9 tokens and 3 occurrences of `search`. What is its background probability? With JM background weight 0.2, what probability does missing `search` receive in C?
6. C has length 2 and no `search`. Calculate its Dirichlet probability for `search` using $\mu = 3$. What does the parameter 3 represent?
7. Suppose $\mu = 20$. Which document gets a larger background mixture weight: a document of length 10 or a document of length 100?

C. Evaluation and interpretation

8. The top three results are relevant, irrelevant, relevant. There are four relevant documents in the full collection. Find P@3, Recall@3, and RR.
9. A query has exactly two relevant documents, appearing at ranks 2 and 4. Calculate AP and RR. Then combine this query with the example from pages 10-11 to calculate MAP and MRR over the two queries.
10. Our smoothed model has MAP = 1 on five teaching queries. What can we conclude? What would we still need before choosing it for a real service?
11. The query is `quantum teleportation`, and both words are absent from the collection. Does collection-based smoothing automatically fix this?
12. Explain the experiment table from your screenshot without using a formula.

A short practice explanation

Complete this sentence aloud:

> We used query likelihood to rank documents by ____. MLE estimates word probabilities from ____. Smoothing helps when ____. We compare against ____ and evaluate the rankings using ____.

14. Answers with reasons

1. **Tokens:** `data`, `data`, `science`. Counts are data: 2 and science: 1. Lowercasing lets `DATA` and `data` match.
2. **Length = 4.** P(search given B) = 1/4. P(data given B) = 3/4. We divide by B's token count because B is the model's source.
3. **(3/4) × (3/4) = 9/16 = 0.5625.** Each query token contributes a factor. Repeated query words are not removed from the likelihood calculation.
4. **C has no `search`, so one factor is zero.** The whole product becomes zero. Logs produce negative infinity for that case. We need smoothing before calculating logs.
5. **P(search given collection) = 3/9 = 1/3.** JM gives C $0.8(0) + 0.2(1/3) = 1/15 \approx 0.0667$.
6. **(0 + 3 × 1/3)/(2 + 3) = 1/5 = 0.2.** The parameter 3 is total background pseudo-count strength. It is not the number of documents, even though our teaching collection happens to have three documents.
7. **The length-10 document.** Its background weight is 20/30, about 0.667. The length-100 document gets 20/120, about 0.167.
8. **P@3 = 2/3, Recall@3 = 2/4, RR = 1.** Precision divides by retrieved positions; recall divides by the total relevant documents.
9. **AP = (1/2 + 2/4)/2 = 0.5; RR = 1/2 = 0.5.** Combined with the earlier query's AP of 2/3 and RR of 1, MAP = 7/12, about 0.5833, and MRR = 0.75.
10. **Relevant documents were ordered before irrelevant documents for those five fully ranked teaching queries.** This is a small result under our labels. We need independent judged data and parameter tuning on separate development queries before a final test on unseen queries.
11. **No.** A word absent from the entire collection has background probability zero. Our notebook reports and drops unknown words. It returns no ranking if the query has no known words. This is an explicit policy.
12. **Possible explanation:** We lowercase and tokenize the text, then compare TF-IDF with query likelihood using three probability estimators. Finally, we check how early relevant documents appear using precision, recall, MAP, and MRR.

Practice sentence: We rank by the likelihood of the query words. MLE uses observed document frequencies. Smoothing helps with query words missing from a document. TF-IDF is our baseline. The metrics judge the resulting order.

15. Glossary and further reading

Term	Plain meaning
Token	One piece of text after tokenization
Vocabulary	The set of distinct tokens
Bag-of-words	Word counts with word order ignored
Language model	A model that assigns probabilities to text
Unigram	A single word; here, modeled without previous-word context
Likelihood	How probable the observed query is under a fixed model
MLE	Probability estimation from observed relative frequencies here
Smoothing	Adjusting estimates to support unseen document terms
Background	Word probabilities estimated from the whole collection
Baseline	A comparison method
Relevant	Useful for the query's intended information need
Parameter	A setting such as JM's 0.2 or Dirichlet's 20
OOV	Out of vocabulary: a term absent from the collection

A few points to remember

Probability of the query and probability of relevance are different. Equal document priors let us rank by query likelihood through Bayes' rule, but the score remains a modeling assumption. **Lower perplexity measures better text prediction**, which does not automatically mean better retrieval. **Smoothing supports missing words, but does not understand synonyms or restore word order.**

Sources and where to continue

- **Course foundation:** DORN Dawin (2026), supplied *Lecture 2: NLP Foundations I: Bag-of-Words and Language Models*, slides 4-8 and 12-20. Read these again after pages 3-8 of this guide.
- **Query likelihood and smoothing:** Manning, Raghavan, and Schütze (2008), *Introduction to Information Retrieval*, Chapter 12.
- **Smoothing research:** Zhai and Lafferty (2001), A Study of Smoothing Methods for Language Models Applied to Ad Hoc IR.
- **Evaluation:** Manning et al., Evaluation of ranked retrieval results.
- **Implementation:** Scikit-learn TfidfVectorizer.

All small numerical examples are worked teaching examples. The page-12 results come from this project's executed notebook and CSV files. The guide's editable source is `learning/learning-guide.qmd`. Render it from the repository root with `quarto render learning/learning-guide.qmd`.